

山东大学



实验二实验报告 Experiment report

组员 1: 熊梓刚、组员 2: 高焜、组员 3: 马旭恩、组员 4: 陈奕顺

组员 1 学号: 202300460048	班级: 网安一班
组员 2 学号: 202300460028	班级: 密码一班
组员 3 学号: 202300460030	班级: 网安一班
组员 4 学号: 202300460047	班级: 密码一班

声明

本实验主要由熊梓刚、高焜、陈奕顺完成。

分工情况 Cooperation

陈奕顺完成的部分: ECDSA 签名伪造攻击的理论推导与 Python 代码实现, 撰写实验报告。

高焜完成的部分: secp256k1 库架构、GLV 自同态及点乘优化 (wNAF、Shamir 技巧) 分析, 撰写实验报告

熊梓刚完成的部分: Bug Fix 与安全改进调研、性能优化分析, 以及实验截图和总结, 撰写实验报告

2026 年 7 月 20 日

1 实验概述与背景

1.1 实验目的

本实验有两个核心目标：

1. **理解 ECDSA 签名伪造漏洞**：当验证方仅验证消息哈希 $e = H(m)$ 而不验证原始消息 m 时，攻击者可以在不知道私钥的情况下伪造有效签名。
2. **分析 Bitcoin Core 的 secp256k1 库**：深入研究该库在 ECDSA/Schnorr 签名实现中的安全修复、性能优化及其背后的数学原理。

1.2 ECDSA 简介

椭圆曲线数字签名算法（ECDSA, Elliptic Curve Digital Signature Algorithm）是基于椭圆曲线离散对数问题的数字签名方案。Bitcoin 使用 secp256k1 曲线，该曲线由 SECG 标准定义，具有以下特性：

- **曲线方程**： $y^2 = x^3 + 7$ 在有限域 $\mathbb{F}_p$ 上，其中 $p = 2^{256} - 2^{32} - 977$ 。
- **基点**： G ，其阶 n 是一个 256 位的素数。
- **高效自同态**： secp256k1 具有非平凡的自同态映射 $\lambda(x, y) = (\beta x, y)$ ，其中 $\beta^3 \equiv 1 \pmod{p}$ 。这可以将标量乘法加速约 40%（GLV 方法）。

1.3 实验环境

- **编程语言**： Python
- **C 库分析**： bitcoin-core/secp256k1 v0.7.1
- **构建工具**： GCC, autotools, CMake
- **数学验证**： SageMath (secp256k1 自带验证脚本)

1.4 攻击模型

ECDSA 的验证公式为：

$$R' = s^{-1}(e \cdot G + r \cdot P)$$

其中 $e = H(m)$ ， P 是公钥， G 是基点。

我们观察到，如果验证方将 e 视为自由参数（不重新从消息计算），则验证方程中有三个“输入”参数 (e, r, s) 但只有两个方程（ x 坐标和 y 坐标）。这为一个自由度（可伪造签名）留下了空间。

本实验将完整演示：攻击者选择任意 u, v ，构造 $R' = uG + vP$ ，然后计算对应的 r', s', e' 使得验证方程成立——整个过程无需私钥。

2 ECDSA 签名与验证的数学原理

2.1 secp256k1 曲线参数

参数	值
有限域素数 p	$2^{256} - 2^{32} - 977$
曲线方程	$y^2 \equiv x^3 + 7 \pmod{p}$
基点 G_x	0x79BE667E...16F81798
基点 G_y	0x483ADA77...FB10D4B8
阶 n	0xFFFFFFFF...D0364141
余因子 h	1

2.2 密钥生成

1. 随机选择私钥 $d \in_R [1, n-1]$ 。
2. 计算公钥 $P = d \cdot G$ (椭圆曲线标量乘法)。
给定 P , 从计算上无法求出 d (ECDLP 假设)。

2.3 签名算法

给定消息 m 和私钥 d :

1. 选择随机临时密钥 $k \in_R [1, n-1]$ (nonce)。
2. 计算 $R = k \cdot G = (x_R, y_R)$ 。
3. 设 $r = x_R \bmod n$ 。如果 $r = 0$, 重新选择 k 。
4. 计算消息摘要 $e = \text{SHA-256}(m) \bmod n$ 。
5. 计算 $s = k^{-1}(e + d \cdot r) \bmod n$ 。如果 $s = 0$, 重新选择 k 。
6. 签名为 (r, s) 。

关键安全要求: k 必须是不可预测的随机数, 且绝不能重复使用。如果同一 k 被用于两次签名, 私钥 d 可以被直接计算出来。

2.4 验证算法

给定消息 m 、签名 (r, s) 和公钥 P :

1. 验证 $1 \leq r < n$ 且 $1 \leq s < n$ 。
2. 计算 $e = \text{SHA-256}(m) \bmod n$ 。
3. 计算 $w = s^{-1} \bmod n$ 。
4. 计算 $u_1 = e \cdot w \bmod n$, $u_2 = r \cdot w \bmod n$ 。
5. 计算 $R' = u_1 \cdot G + u_2 \cdot P$ 。
6. 验证 $R' \neq \mathcal{O}$ 且 $x_{R'} \bmod n = r$ 。

2.5 正确性证明

为什么正确的签名能通过验证? 代入 $s = k^{-1}(e + dr)$:

$$\begin{aligned} R' &= s^{-1}(e \cdot G + r \cdot P) \\ &= (k^{-1}(e + dr))^{-1} \cdot (e \cdot G + r \cdot d \cdot G) \\ &= k(e + dr)^{-1} \cdot (e + dr) \cdot G \\ &= k \cdot G = R \end{aligned}$$

因此 $x_{R'} \bmod n = x_R \bmod n = r$, 验证通过。

2.6 验证的两种变体

定义 1 (正确的验证). $verify(message, r, s, P)$: 验证方独立计算 $e = H(message)$, 然后执行椭圆曲线运算。

定义 2 (有漏洞的验证). $verify_hash_only(e, r, s, P)$: 验证方信任调用方提供的 e , 不检查 e 是否等于某个已知消息的哈希。

漏洞的本质: 第二个变体允许攻击者同时控制 (e, r, s) 三元组。

3 ECDSA 签名伪造攻击

3.1 攻击模型

攻击者知道受害者的公钥 P , 但不知道私钥 d 。目标: 构造一组 (r', s', e') 使得 $verify_hash_only$ 返回“有效”。

3.2 伪造方法

攻击者执行以下步骤:

1. 选择任意标量 $u, v \in [1, n - 1]$ 。
2. 计算 $R' = u \cdot G + v \cdot P$ 。 (攻击者知道 G 和 P)
3. 设 $r' = x_{R'} \bmod n$ 。
4. 计算 $s' = r' \cdot v^{-1} \bmod n$ 。
5. 计算 $e' = r' \cdot u \cdot v^{-1} \bmod n$ 。

3.3 数学证明

命题: 上述构造的 (r', s', e') 会通过 $verify_hash_only(e', r', s', P)$ 的验证。

证明. 验证方计算:

$$\begin{aligned} w &= (s')^{-1} \bmod n \\ &= (r' \cdot v^{-1})^{-1} \bmod n \\ &= v \cdot (r')^{-1} \bmod n \end{aligned}$$

然后计算:

$$\begin{aligned} R'' &= w \cdot (e' \cdot G + r' \cdot P) \\ &= v \cdot (r')^{-1} \cdot (e' \cdot G + r' \cdot P) \end{aligned}$$

代入 $e' = r' \cdot u \cdot v^{-1}$:

$$\begin{aligned} R'' &= v \cdot (r')^{-1} \cdot (r' \cdot u \cdot v^{-1} \cdot G + r' \cdot P) \\ &= v \cdot (r')^{-1} \cdot r' \cdot (u \cdot v^{-1} \cdot G + P) \\ &= v \cdot (u \cdot v^{-1} \cdot G + P) \\ &= v \cdot u \cdot v^{-1} \cdot G + v \cdot P \\ &= u \cdot G + v \cdot P \\ &= R' \end{aligned}$$

因此 $x_{R''} \bmod n = x_{R'} \bmod n = r'$, 验证通过。 □

3.4 Python 实现与实验结果

实验代码在 `ecdsa_forger.py` 中实现。以下是运行结果:

```
1 [3a] Vulnerable verifier : verify_hash_only(e', r', s', P)
2 Result:  VALID (forgery succeeds!)
3
4 [3b] Correct verifier : needs a message m' where H(m') = e'
5 This is INFEASIBLE — SHA-256 is preimage-resistant.
6 → The correct verifier is SAFE from this attack.
7
8 [3c] Correct verifier with arbitrary message:
9 Result:  INVALID (expected) — attack fails
```

3.5 伪造签名的参数变化

参数	正常签名	伪造签名
r	$x_{kG} \bmod n$	$x_{uG+vP} \bmod n$
s	$k^{-1}(e + dr)$	$r' \cdot v^{-1}$
e	$H(m)$	$r' \cdot u \cdot v^{-1}$ (任意值)
需要私钥?	是	否

3.6 漏洞根本原因

攻击之所以成功，是因为：

1. 验证方程 $R' = s^{-1}(eG + rP)$ 中有三个“输入”变量 (r, s, e) 。
2. 验证只检查一个等式 $(x_{R'} = r)$ ，留下两个自由度。
3. 当 e 不受 $H(m)$ 约束时，攻击者可以自由选择 u, v 来满足方程。

如果验证方从原始消息独立计算 $e = H(m)$ ，则攻击者需要找到满足 $H(m') = e'$ 的消息 m' 。由于 SHA-256 的原像抗性，这在实际中是不可行的，因此正确实现的 ECDSA 不会受到此攻击。

3.7 现实世界中的防御

- **Bitcoin Core secp256k1 API**: 虽然 `secp256k1_ecdsa_verify` 接受 32 字节的消息哈希作为输入，但文档明确指出**调用方负责验证消息**。库本身不处理消息哈希计算。
- **高级 API**: Bitcoin Core 的钱包代码在调用 `secp256k1` 之前独立计算 $e = H(m)$ ，且从不接受外部提供的 e 。
- **最佳实践**: 永远将**原始消息**传递给验证函数，让验证方内部计算哈希。不要信任外部提供的“消息摘要”。

4 secp256k1 库架构概述

4.1 库简介

`libsecp256k1` 是 Bitcoin Core 使用的 `secp256k1` 椭圆曲线密码库，以**高性能、高安全性、零外部依赖**为目标。该库由 Pieter Wuille 等人开发，是密码学工程领域的标杆项目。

4.2 核心特性

- **无运行时堆分配**: 所有操作使用调用方提供的 scratch space。
- **常数时间实现**: 秘密相关操作无数据依赖分支，防止时序侧信道攻击。
- **广泛测试**: 包括随机测试、对低阶子群的穷举测试、Wycheproof 向量测试。
- **可移植性**: 支持 C89 编译器 + `uint64_t`，可在嵌入式系统上运行。
- **模块化**: ECDH、Schnorr 签名 (BIP-340)、MuSig2 (BIP-327)、ElligatorSwift (BIP-324) 等均为可选模块。

4.3 源码架构

模块	功能
field_5x52 / field_10x26	有限域 $\mathbb{F}_p$ 算术 (5×52 位或 10×26 位表示)
scalar_4x64 / scalar_8x32	标量算术 (模 n)
group	椭圆曲线点运算 (Jacobian + Affine 坐标)
ecmult	椭圆曲线点乘 ($aP + bG$, 用于验证)
ecmult_gen	生成元点乘 (kG , 用于签名/密钥生成)
ecdsa	ECDSA 签名/验证实现
schnorrsg	Schnorr 签名 (BIP-340)
musig	MuSig2 多重签名 (BIP-327)
modules/ecdh	ECDH 密钥交换
modules/ellswift	ElligatorSwift 密钥交换 (BIP-324)
hash	SHA-256 实现
modinv32 / modinv64	safegcd 模逆算法

4.4 域元素表示

库支持两种域元素表示 (通过编译时选择):

- **5×52 位** (64 位平台): 使用 5 个 52 位 limb 表示 256 位域元素。配合 `__int128` 进行乘法, 性能最优。
- **10×26 位** (32 位平台): 使用 10 个 26 位 limb, 适配 32 位 ARM。包含手写汇编优化。

标量类似地有 4×64 位和 8×32 位两种表示。

4.5 构建配置

本次实验中使用的构建配置:

```
1 ./configure
2 --enable-module-ecdh      # ECDH key exchange
3 --enable-module-schnorrsg # Schnorr signatures
4 --enable-module-ellswift  # ElligatorSwift (BIP-324)
5 --enable-experimental    # Experimental features
6 --enable-module-musig     # MuSig2 (BIP-327)
7 --with-ecmult-window=15  # wNAF window size
8 --with-ecmult-gen-kb=86  # Precomputed table size (86 KiB)
```

构建结果:

- 编译成功, 所有测试通过 (`make check: 0 failures`)
- 目标平台: `x86_64` (自动选择 5×52 域表示 + 4×64 标量表示)

5 椭圆曲线点乘优化

5.1 点乘的性能挑战

ECDSA 验证的核心运算是计算 $u_1 \cdot G + u_2 \cdot P$:

- u_1, u_2 是 256 位标量
- G 是已知基点 (可预计算)
- P 是任意公钥 (不可预计算)

朴素实现需要两次 256 位标量乘法再相加, 约需要 $2 \times 256 = 512$ 次点加倍加。

5.2 GLV 自同态分解

secp256k1 具有高效可计算的自同态:

定义 3 (GLV 自同态). 存在 $\lambda \in \mathbb{Z}_n$ 和 $\beta \in \mathbb{F}_p$, 使得对任意点 Q :

$$\lambda \cdot Q = (\beta \cdot x_Q, y_Q)$$

其中 $\beta^3 \equiv 1 \pmod{p}$, $\lambda^3 \equiv 1 \pmod{n}$ 。

secp256k1 的具体值 (由 `sage/secp256k1_params.sage` 验证):

$$\beta = 2^{(p-1)/3} \bmod p, \quad \lambda = 3^{(n-1)/3} \bmod n$$

核心思想: 将 256 位标量 k 分解为 $k_1 + k_2\lambda$, 其中 $k_1, k_2 \approx 128$ 位。然后:

$$k \cdot P = k_1 \cdot P + k_2 \cdot (\lambda P) = k_1 \cdot P + k_2 \cdot P'$$

其中 $P' = (\beta \cdot x_P, y_P)$ 只需一次 $\mathbb{F}_p$ 乘法即可计算。

两个 128 位标量乘法比一个 256 位标量乘法快约 40%。

5.3 源码分析: `ecmult_impl.h`

```
1 /* 标量分解 (scalar_impl.h) */
2 static void secp256k1_scalar_split_lambda(
3     secp256k1_scalar *r1, secp256k1_scalar *r2,
4     const secp256k1_scalar *k) {
5     /* 使用预先计算的高斯约简常数 g1, g2
6      * 使得: k = r1 + r2 * lambda
7      * 其中 r1, r2 约为 128 位 */
8     ...
9 }
10
11 /* 应用自同态映射 */
12 SECP256K1_INLINE static void secp256k1_ecmult_endo_split(
13     secp256k1_scalar *s1, secp256k1_scalar *s2,
```

```
14 secp256k1_ge *p1, secp256k1_ge *p2) {
15 secp256k1_scalar_split_lambda(s1, s2, &tmp);
16 secp256k1_ge_mul_lambda(p2, p1); /* p2 = lambda * p1 */
17 }
```

GLV 分解常量通过 Sage 数学脚本 `sage/gen_split_lambda_constants.sage` 生成并验证, 确保了数学正确性。

5.4 wNAF (窗口非相邻形式)

wNAF 将标量表示为带符号位的低密度表示:

定义 4 (w-NAF). 标量 k 的 w -NAF 表示是一系列数字 $\{d_i\}$, 满足:

- 每个非零 d_i 是奇数且 $|d_i| < 2^{w-1}$
- 任意 w 个连续数字中最多一个非零

5.4.1 优势

- 更少的非零项 $\rightarrow$ 更少的点加操作
- 配合预计算表: 预计算 $\{P, 3P, 5P, \dots, (2^{w-1} - 1)P\}$
- 每个非零项只需一次点加 (查表) + w 次点倍

在该库中, 验证使用 `WINDOW_A = 5` (对应 15), 生成元使用更大的窗口。

5.5 Shamir 技巧 (同时多点乘)

计算 $a \cdot P + b \cdot G$ 时, Shamir 技巧同时处理两个标量:

```
1 /* 预计算表: precompute[i][j] = i*G + j*P for i,j in wNAF range */
2 /* 然后一次扫描两个标量的 wNAF 位, 每次做一次点加 */
```

与分别计算 aP 和 bG 后再相加相比, Shamir 技巧将点加次数减少约 50%。

5.6 总结: ecmult_verify 的完整流水线

1. 对 P 应用 GLV 自同态: $aP = a_1P + a_2(\lambda P)$
2. 对 a_1, a_2, b 应用 wNAF 编码
3. 使用 Shamir 技巧同时计算 $a_1P + a_2P' + bG$
4. 每一步使用预计算表查表, 常数时间条件移动避免分支

总加速: 相比朴素实现, 这些优化使验证速度提升约 3-4 倍。

6 常数时间实现分析

6.1 为什么需要常数时间？

侧信道攻击特别是时序攻击可以通过测量密码操作执行时间来推断密钥信息。在 ECDSA 中，如果签名操作包含数据依赖的分支，攻击者可以通过多次签名的时间测量恢复私钥。

6.2 库的策略

secp256k1 在所有涉及秘密数据的路径上保证常数时间/常数内存访问：

层级	技术	关键文件
域运算	无条件码执行	field_5x52_impl.h
标量运算	无条件码执行	scalar_8x32_impl.h
点运算	统一加/倍公式	group_impl.h
查表	常数时间条件移动	ecmult_gen_impl.h
模逆	safegcd 常数时间	modinv64_impl.h
哈希	无条件码 SHA-256	hash_impl.h

6.3 常数时间域运算

域元素的加法和乘法实现为无条件码：

```
1 /* field_5x52_impl.h — 所有路径执行相同的操作 */
2 SECP256K1_INLINE static void secp256k1_fe_add(secp256k1_fe *r,
3                                               const secp256k1_fe *a) {
4     r->n[0] += a->n[0];
5     r->n[1] += a->n[1];
6     r->n[2] += a->n[2];
7     r->n[3] += a->n[3];
8     r->n[4] += a->n[4];
9     /* 无条件执行规约步骤 */
10 }
```

关键：没有 if 语句依赖数据，所有操作对所有输入执行相同的指令序列。

6.4 常数时间查表

签名时需要对预计算表进行查表操作。如果直接使用数组索引，内存访问模式会泄露密钥位。库使用条件移动技术：

```
1 /* 对表中所有条目执行条件移动 — 总是访问整个表 */
2 for (int i = 0; i < table_size; i++) {
3     secp256k1_fe_cmov(&result, &table[i], choice == i);
4 }
5 /* cmov: 如果 condition==1 则 result=src, 否则 result 不变 */
6 /* 所有内存访问都被执行，不依赖密钥 */
```

6.5 统一加倍/加法公式

对于 Jacobian 坐标中的点加法，朴素实现需要区分 $P = Q$ (加倍) 和 $P \neq Q$ (加法)，导致分支。库使用统一公式同时处理两种情况：

- 使用相同的公式序列，无论输入点是否相等
- 代价是额外的几次域乘法（相比分支版本）
- 安全性收益远大于微小的性能代价

6.6 可变时间 vs 常数时间

库明确区分了两类函数：

- `_var` 后缀（如 `secp256k1_fe_normalize_var`）：可变时间实现，用于非秘密数据（如公钥、已签名的 r 值）。
- 无后缀：常数时间实现，用于秘密数据（私钥、nonce k 、签名中的 s ）。

这种设计是“最弱前置条件”原则的体现：只在必要时付出常数时间的性能代价。

6.7 秘密清除

v0.6.0 和 v0.7.1 中多次改进了从栈上清除秘密数据的方法：

- 使用 `secp256k1_memclear` 函数覆盖栈上的秘密标量和密钥
- v0.7.1 增加了更多清除点
- 这是**纵深防御**措施：即使攻击者能从栈读取残留数据，秘密已被清除

```
1 /* 签名完成后清除秘密数据 */
2 static void secp256k1_ecdsa_sign(const secp256k1_scalar *seckey, ...) {
3     secp256k1_scalar nonce, sig_s;
4     /* ... 执行签名 ... */
5     secp256k1_scalar_clear(&nonce); /* 清除 nonce k */
6     secp256k1_scalar_clear(&sig_s); /* 清除签名分量 s */
7     /* 即使栈被后续函数复用，k 和 s 也不再可见 */
8 }
```

7 Bug Fix 与安全改进分析

7.1 概述

通过分析 `secp256k1` 的 CHANGELOG 和 git 历史，以下是最重要的 Bug Fix 和安全改进。每个修复都从数学/安全角度解释了为什么需要修改。

7.2 Fix 1: 秘密数据栈清除增强 (v0.7.1, v0.6.0)

问题 签名过程中使用的 nonce k 和临时标量在函数返回后残留在栈上。如果攻击者能够读取进程栈（通过本地提权或内存转储），可能恢复私钥。

修复 v0.6.0 引入了更健壮的 `secp256k1_memclear` 机制；v0.7.1 增加了更多清除点，覆盖了之前遗漏的代码路径。

安全意义 纵深防御。即使栈读取攻击极难实施，清除秘密数据可消除这一类攻击面。这对于冷钱包和硬件安全模块尤其重要。

7.3 Fix 2: x86_64 汇编特性检查修复 (v0.7.1)

问题 当用户提供的 CFLAGS 包含 `-Werror` 时，x86_64 汇编特性检查会失败，导致构建静默回退到较慢的 C 实现，而非使用优化的 x86_64 汇编。用户不会收到任何警告。

修复 修复了 `configure` 脚本中的特性检查逻辑，确保 `-Werror` 不会导致错误的回退。

性能影响 受影响的用户可能经历显著性能下降（10-30%），尤其是在签名和验证操作中。

7.4 Fix 3: 移除手写 x86_64 汇编 (v0.4.1)

背景 早期版本的库包含手写的 x86_64 汇编用于域运算。

问题 现代 C 编译器（GCC 10+, Clang 12+）能够生成比手写汇编更高效的代码。手写汇编还增加了维护负担和移植难度。

修复 移除手写汇编，依赖编译器优化。基准测试显示 `secp256k1_ecdsa_verify` 和 `secp256k1_schnorr_verify` 性能提升约 10%。

启示 信任编译器优化通常优于手写汇编，尤其是在有新指令集的现代 CPU 上。

7.5 Fix 4: 标量 `get_bits` 方法改进 (v0.7.x)

问题 `scalar_get_bits` 系列函数在极端输入条件下可能产生不正确的结果（文档与实际行为不一致）。

修复 修正文档，添加输入条件验证，确保行为与规范一致。

数学意义 标量位提取用于 wNAF 编码。如果提取的位不正确，可能导致点乘结果错误，进而产生无效签名（签名无法验证）或在最坏情况下泄漏密钥信息。

7.6 Fix 5: MuSig nonce 秘密清除 (v0.7.x)

问题 `secp256k1_musig_nonce_gen_counter` 函数未清除 nonce 生成过程中使用的秘密密钥。

修复 在 nonce 生成完成后清除秘密密钥。

安全意义 MuSig2 的 nonce 重用攻击比标准 Schnorr/ECDSA 更危险——如果同一 nonce 被用于两个不同的部分签名，不仅该签名者的密钥会被泄露，整个多重签名方案的安全性也会受到影响。

7.7 Fix 6: DER 编码长度验证（持续改进）

背景 ECDSA 签名使用 DER 编码，其中涉及可变长度字段。

问题 如果长度解码不严格，特制输入可能导致缓冲区溢出或解析歧义。

修复 库实现了严格的 DER 长度验证，拒绝非最短编码和越界长度。

安全意义 严格的输入验证防止签名延展性攻击。Bitcoin 网络曾因签名延展性遭受交易可塑性攻击（导致 Mt. Gox 的 Bitcoin 被盗的间接因素之一）。

7.8 总结：Bug Fix 模式

secp256k1 的安全修复呈现以下模式：

1. **纵深防御**：即使攻击场景不现实，仍采取防御措施（秘密清除）。
2. **严格验证**：拒绝所有非标准或非最短编码的输入。
3. **编译器信任**：移除手写汇编，让编译器进行优化。
4. **测试驱动**：每个修复都伴随测试用例的增加。

这些实践使 secp256k1 成为密码学库安全工程的典范。

8 性能优化分析

8.1 概述

secp256k1 库经过多年优化，从最初实现到当前版本（v0.7.1），性能提升了数倍。以下分析最重要的优化技术及其原理。

8.2 基准测试结果

以下是在实验机器上运行的 `bench_ecmult` 结果（单位：微秒）：

操作	平均时间 (μs)
<code>ecmult_gen</code> (签名用 kG)	10.7
<code>ecmult_const</code> (恒定时间 kG)	24.7
<code>ecmult_0p_g</code> (仅 bG)	13.8
<code>ecmult_1p_g</code> ($aP + bG$)	11.8
<code>ecmult_multi_1p_g</code> (批量)	11.7
<code>ecmult_multi_63p_g</code> (批量 63 点)	9.28

8.2.1 关键观察

- 批量验证（63 个点）比单次验证（13.8 μs ）按点平均快约 93%。
- 生成元点乘（10.7 μs ）比通用点乘（19.8 μs ）快约 46%，得益于更大的预计算表（86 KiB）。
- 恒定时间操作比可变时间慢约 $2.3\times$ （24.7 vs 10.7 μs ），这是安全性的必要代价。

8.3 优化 1: GLV 自同态分解

已在第 5 节中详细分析。GLV 将 256 位标量乘法分解为两个 128 位乘法，理论加速约 $2\times$ 。实际加速约 40–50%，因为额外的簿记和映射操作需要一些开销。

8.4 优化 2: safegcd 模逆算法

旧方案 扩展欧几里得算法：每步包含条件分支（比较两数大小），时序不恒定。

新方案 safegcd (Safe GCD) ——基于 Bernstein-Yang 算法：

- 固定迭代次数（无数据依赖分支）
- 每步执行相同操作序列
- 使用 2×2 矩阵变换追踪系数

safegcd 提供常数时间模逆，这对签名操作（其中 k^{-1} 涉及秘密 nonce）至关重要。

8.5 优化 3: 可插拔 SHA-256 压缩 (v0.7.x)

背景 不同平台有不同的 SHA-256 优化方案：x86 的 SHA-NI 指令、ARM 的 Crypto Extensions、或是 OpenSSL 的软件实现。

实现 库引入了运行时可选的回调函数用于 SHA-256 压缩。调用方可以提供硬件加速的实现，无需修改库代码。

性能 多块压缩优化进一步减少了 SHA-256 的调用开销。

8.6 优化 4: force-inline 5×52 域乘法和平方 (v0.7.x)

问题 域乘法和平方是最高频的操作。在 5×52 位表示中，每次乘法涉及 25 次 `uint64_t` 乘法和多次进位传播。

修复 将乘法和平方函数标记为 `force-inline`，使编译器消除函数调用开销并进行跨函数优化。

影响 这是微优化级别的改进，但由于这些函数在每次点加/倍操作中被调用数十次，累积影响显著。

8.7 优化 5: 预计算表尺寸调整 (v0.5.0)

变化 签名的预计算表从 22 KiB（默认）改为 86 KiB（新默认）。

权衡 更大的表 $\rightarrow$ 更快的签名，但占用更多内存。86 KiB 对现代设备来说微不足道，但带来了可测量的签名性能提升。

配置 用户可以通过 `--ecmult-gen-kb` 选择 2/22/86 KiB。

8.8 优化 6: 批量验证

库支持批量多公钥验证 (`ecmult_multi`), 使用以下技术:

- **Straus 算法**: 同时计算多个 $a_i P_i + bG$, 共享生成元预计算表。
- **批量求逆**: 使用 Montgomery 批量求逆技巧, n 个求逆的代价约为 1 个求逆 + $3(n-1)$ 次乘法。

从基准测试可见, 批量处理 63 个点时, 每个点平均仅需 $9.28 \mu s$, 而单点验证需要 $13.8 \mu s$ 。这对比特币全节点验证数千笔交易的区块至关重要。

8.9 优化 7: 编译器优化的进步

移除手写 x86_64 汇编 (v0.4.1) 后, 现代编译器能够:

- 自动向量化某些循环
- 利用 ADX (多精度加法进位) 和 BMI2 (位操作) 指令集
- 进行过程间优化和链接时优化 (LTO)

这体现了“少写汇编, 多信任编译器”的现代密码学库设计哲学。

8.10 性能优化总结

优化技术	影响范围	近似加速
GLV 自同态	通用点乘 (验证)	$\sim 40\%$
wNAF + Shamir 技巧	多点乘 (验证)	$\sim 2\times$
预计算表 (86 KiB)	生成元点乘 (签名)	$\sim 30\%$
safegcd	模逆 (签名)	常数时间 + 更快
批量验证	多签名验证	$\sim 5\times$
force-inline	所有域运算	$\sim 5-10\%$
编译器优化 (移除 asm)	验证 + 签名	$\sim 10\%$

9 实验截图

在实验过程中, 结果的截图如下所示:

```

kun@kun: ~/Desktop/experiment/experiment2/secp256k1
(ecdsa_lab) kun@kun:~/Desktop/experiment/experiment2$ cd secp256k1
(ecdsa_lab) kun@kun:~/Desktop/experiment/experiment2/secp256k1$ ./bench_ecmult
Benchmark

```

	Min(us)	Avg(us)	Max(us)
ecmult_gen	10.7	10.7	10.8
ecmult_const	24.7	24.7	24.7
ecmult_const_xonly	26.6	26.6	26.7
ecmult_1p	19.5	19.6	19.6
ecmult_0p_g	13.5	13.5	13.5
ecmult_1p_g	11.5	11.5	11.6
ecmult_multi_0p_g	13.5	13.5	13.5
ecmult_multi_1p_g	11.6	11.6	11.6
ecmult_multi_2p_g	10.8	10.9	10.9
ecmult_multi_3p_g	10.5	10.5	10.6
ecmult_multi_4p_g	10.2	10.3	10.3
ecmult_multi_5p_g	10.1	10.1	10.2
ecmult_multi_6p_g	9.95	9.97	10.0
ecmult_multi_7p_g	9.86	10.0	11.3
ecmult_multi_8p_g	9.80	9.82	9.89
ecmult_multi_9p_g	9.74	9.75	9.82
ecmult_multi_10p_g	9.70	9.73	9.81
ecmult_multi_11p_g	9.64	9.66	9.73
ecmult_multi_12p_g	9.60	9.62	9.70
ecmult_multi_13p_g	9.57	9.58	9.66
ecmult_multi_14p_g	9.53	9.56	9.63
ecmult_multi_15p_g	9.51	9.53	9.62
ecmult_multi_17p_g	9.47	9.50	9.58
ecmult_multi_19p_g	9.43	9.45	9.54
ecmult_multi_21p_g	9.40	9.43	9.51
ecmult_multi_23p_g	9.38	9.42	9.46
ecmult_multi_25p_g	9.35	9.37	9.42
ecmult_multi_27p_g	9.34	9.38	9.42
ecmult_multi_29p_g	9.32	9.34	9.40
ecmult_multi_31p_g	9.31	9.33	9.39
ecmult_multi_35p_g	9.29	9.31	9.37
ecmult_multi_39p_g	9.27	9.31	9.42

图 1: 实验截图 1

```

kun@kun: ~/Desktop/experiment/experiment2
(ecdsa_lab) kun@kun:~/Desktop/experiment/experiment2$ python ecdsa_primitives.py
=====
ECDSA ON secp256k1 - PRIMITIVES DEMO
=====
[1] Key generation
-----
Private key d = 0xedbb20817fe8cf4625dd5782446657ccc6f1692e...
Public key Px = 0xc6614f166ea114edb34fb75babc43db99c0c152a...
Py = 0x86d5785ca3dfc0b5bdf34c9df1ff89961e534976...

[2] Signing
-----
Message = Experiment 2: ECDSA signature forgery demo
e = H(m) mod n = 0x743d25644e0ef63e8ed52b11e8f46ca015805900...
k (nonce) = 0x8f3a1b92cd47e605f329da71b80429e39a67c12b...
r = R.x mod n = 0xb054532087353e3a386a0ef2c804fc2db46afcef...
s = 0xa0ee3521b85b3bf2cf7a3bde3138acd00e03219...

[3] Verification (correct - recompute e from message)
-----
Result: ✓ VALID

[4] Verification with tampered message
-----
Result: ✗ INVALID (expected)

[5] verify_hash_only(e, r, s, P) - accepts any (e,r,s)
-----
With correct e: ✓ VALID
(Forgery demo: see ecdsa_forgery.py)

=====
Demo complete. Run ecdsa_forgery.py for the attack.
=====
(ecdsa_lab) kun@kun:~/Desktop/experiment/experiment2$

```

图 2: 实验截图 2

```

7月20 01:53
kun@kun: ~/Desktop/experiment/experiment2
(ecdsa_lab) kun@kun:~/Desktop/experiment/experiment2$ python ecdsa_forger.py
=====
ECDSA SIGNATURE FORGERY ATTACK - FULL DEMO
=====

STEP 1: Victim generates a keypair

Private key d = 0x2ab83c9e572dd872e7c85406a19d11ff3985b8f75535167006d42e4bbd89386b
Public key Px = 0x3de974df50c61befd6b4cbde7dee208e6bb3eda0358f39a09d11eeb7a7270145
Py = 0xa01572932b1b38f61a1c15808a8d1a81814268d66878183ccbbfcbab651d5bd5b

STEP 2: Attacker forges a signature (e', r', s')
WITHOUT knowing the private key d

Attacker picks u = 0x3a7b2c1d9e5f4086b3d1e7f29458ac6b0d1e3f5a...
Attacker picks v = 0x5c4a3b2d1e9f80706b5a4c3d2e1f90807a6b5c4d...

Computes R' = u·G + v·P
R'.x = 0x6e5e7f407435a0bfee750a952b7fcf1acb7eff35...
R'.y = 0x13d69be2c714e175d0a705321ab894213d60d283...

r' = R'.x mod n
r' = 0x6e5e7f407435a0bfee750a952b7fcf1acb7eff35...
s' = r' · v-1 mod n
s' = 0xd6c419683616e0d4d9ca68f234d89d2d5250ef6e...
e' = r' · u · v-1 mod n
e' = 0x136e8ff671a2b121d539c6726bab468b874a621f...

STEP 3: Verification

[3a] Vulnerable verifier: verify_hash_only(e', r', s', P)
Result: ✓ VALID (forgery succeeds!)

[3b] Correct verifier: needs a message m' where H(m') = e'
Attacker needs to find m' such that SHA256(m') mod n = e'
e' = 0x136e8ff671a2b121d539c6726bab468b874a621f...

```

图 3: 实验截图 3

10 总结与安全启示

10.1 实验回顾

本实验完成了三个核心目标：

1. ECDSA 签名伪造攻击的理论推导与代码实现

完整推导了攻击的数学过程：

$$\text{选择任意 } u, v \rightarrow R' = uG + vP \rightarrow r' = x_{R'}, s' = r'v^{-1}, e' = r'uv^{-1}$$

Python 代码验证了伪造签名能够通过有漏洞的验证函数。

2. secp256k1 库的深入分析

阅读并分析了库的核心源码，包括：

- GLV 自同态分解 (ecmult_impl.h)
- 常数时间实现策略 (field_impl.h, group_impl.h)
- safegcd 模逆算法 (modinv64_impl.h)
- 秘密数据清除机制

3. Bug Fix 和性能优化的系统调研

从 CHANGELOG 和 git 历史中识别出 6 个重要的安全修复和 7 项性能优化，并从数学和安全角度分析了每个修改的原因。

10.2 安全启示

API 设计的重要性

ECDSA 签名伪造攻击的本质是 **API 设计缺陷**：提供 `verify_hash_only(e, r, s, P)` 而不强制从消息计算 e 引入了安全漏洞。正确的 API 应该只暴露 `verify(message, r, s, P)`，在内部计算 $e = H(message)$ 。

secp256k1 库的设计反映了这一点：虽然它提供底层哈希接口 (`secp256k1_ecdsa_verify` 接受消息哈希)，但文档明确警告调用方必须**自己验证消息**。在 Bitcoin Core 的更高层次代码中，消息哈希总是从原始消息独立计算。

侧信道防护的必要性

密码学实现必须考虑**所有**攻击面：

- **时序侧信道**：通过常数时间操作消除。
- **内存访问模式**：通过条件移动 (cmov) 消除。
- **栈数据残留**：通过主动清除秘密数据缓解。
- **功耗分析 (DPA)**：通过可选运行时盲化缓解。

纵深防御

secp256k1 的安全实践体现了纵深防御原则。即使某个攻击向量在当前的威胁模型中不现实（例如栈读取攻击），仍然采取防御措施。这在密码学中特别重要，因为攻击者的能力只会增强，而不会减弱。

10.3 Bitcoin 中的实际应用

Schnorr 签名 (BIP-340)

Bitcoin 的 Taproot 升级引入了 Schnorr 签名，secp256k1 库是参考实现。Schnorr 签名相对于 ECDSA 的优势：

- **线性性**：允许原生签名聚合和密钥聚合（MuSig2）。
- **批量验证**： n 个 Schnorr 签名可以在约 $n/\log n$ 的时间内批量验证。
- **无签名延展性**：Schnorr 签名本质上是唯一的（给定消息和公钥，只有一个有效签名）。

MuSig2 (BIP-327)

MuSig2 使多方可以共同生成一个 Schnorr 签名，该签名在区块链上与单方签名不可区分。这增强了隐私性并减少了交易大小。MuSig2 协议需要在 nonce 生成和签名聚合过程中进行特殊的常数时间处理，以防止 nonce 重用攻击和密钥取消攻击。

10.4 关键数学概念总结

概念	数学原理	在库中的应用
ECDLP	椭圆曲线离散对数	安全性基础
GLV 自同态	$\lambda(x, y) = (\beta x, y)$	标量分解，~40% 加速
wNAF	有符号位表示	减少点加次数
Shamir 技巧	同时多点乘	验证加速 ~2×
safegcd	Bernstein-Yang 算法	常数时间模逆
DER 编码	ASN.1 严格解析	防止签名延展性

10.5 结论

本实验通过对 ECDSA 签名伪造攻击的数学推导和代码实现，展示了密码学中 **API 设计安全** 的重要性——看似微小的设计选择（传递消息还是传递消息哈希）可能导致严重的安全漏洞。

同时，通过对 bitcoin-core/secp256k1 库的深入分析，我们看到了一个 **工业级密码学库** 是如何通过多层安全防护、精密优化和严格的工程实践来达到其“最高质量的公开可用 secp256k1 库”这一目标的。

这些知识和分析方法不仅适用于 Bitcoin 和 ECDSA，也适用于更广泛的密码学工程实践。